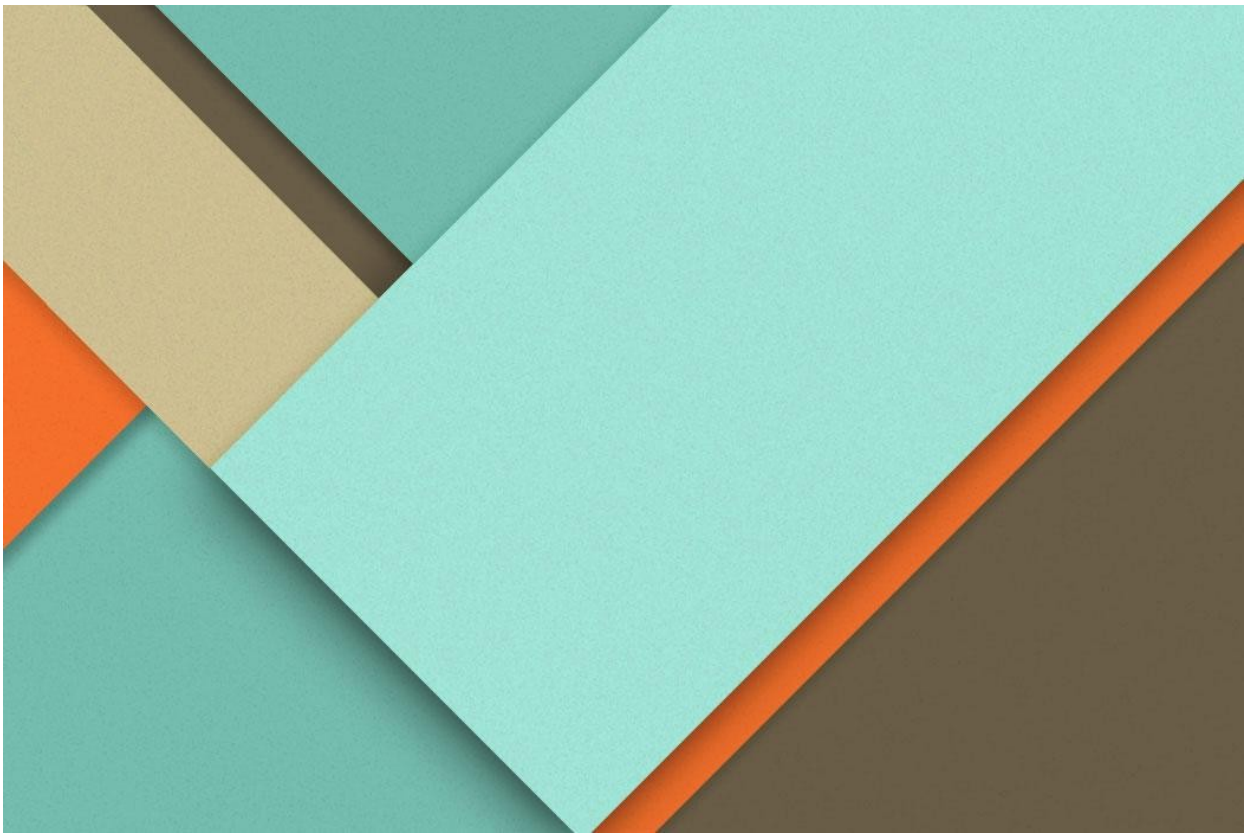


California Housing · Práctica 2

05/10/2026

Alejandro Gracia

Dan Bolocan

CPIFP Los Enlaces

2º DAW diurno

1) Comparación de plataformas cloud [RA2.A]

Se comparan Render y Railway para desplegar la aplicación como servicio web con URL pública.

	Render	**Railway**
Facilidad de uso	Muy alta: se conecta el repositorio de GitHub, se indican el Build Command y el Start Command y despliega solo en cada push.	Alta: se despliega desde GitHub y detecta el proyecto automáticamente, pero la interfaz (servicios, variables, dominio público) es algo menos guiada.
Plan gratuito	Plan Free: 750 horas de instancia al mes por workspace, una sola instancia.	Prueba de 30 días con 5 $ de crédito único; después, plan Free con 1 $ de crédito al mes que no se acumula.
Límites principales	El servicio se suspende tras 15 min sin tráfico y tarda cerca de 1 min en reactivarse; sin disco persistente ni acceso SSH.	Prueba limitada a 1 GB de RAM y CPU compartida; al agotarse el crédito el servicio se detiene. No se documenta suspensión por inactividad.

Conclusión: se usa Render porque es el que pide la práctica, su plan gratuito no caduca y la configuración es más sencilla; su inconveniente es el arranque en frío tras la suspensión. Fuentes: documentación oficial de Render (render.com/docs/free) y de Railway (docs.railway.com, free trial), consultadas el 05/10/2026.

2) Selección y configuración de la plataforma [RA2.B]

Se ha creado una cuenta en Render iniciando sesión con GitHub y se ha subido el proyecto a un repositorio público que contiene el código que genera el modelo (model_training.py y housing.csv), el modelo entrenado (modelo_california.pkl), app.py, requirements.txt y README.md.

Repositorio de GitHub: https://github.com/<usuario>/<repositorio>

[PENDIENTE: captura del repositorio de GitHub (con el nombre del autor y P2)]

[PENDIENTE: captura del repositorio conectado en Render]

[PENDIENTE: captura de la configuración del servicio en Render (plan Free, Build Command, Start Command)]

3) Instalación y configuración de dependencias [RA2.C]

requirements.txt contiene solo lo necesario para ejecutar app.py en producción, con versiones fijadas para que el modelo, serializado con scikit-learn 1.9.1, se cargue igual que se entrenó: fastapi y uvicorn (servidor web), requests (descarga del modelo), pandas y numpy (entrada del modelo), scikit-learn, scipy y joblib (carga y ejecución del modelo). No se incluye matplotlib porque solo se usaba para las gráficas de la práctica 1. Recurso: requirements.txt (también en el repositorio).

4) Despliegue de la aplicación de IA [RA2.D]

Se ha desplegado app.py (FastAPI) en Render con el comando de arranque uvicorn app:app --host 0.0.0.0 --port $PORT. GET / devuelve {"status": "ok"} y POST /predict devuelve el precio medio estimado (MedHouseVal, en cientos de miles de dólares).

URL pública del servicio: https://<tu-servicio>.onrender.com

[PENDIENTE: captura de la respuesta de GET / (health-check)]

[PENDIENTE: captura de una predicción en POST /predict]

5) Pruebas de funcionalidad y rendimiento [RA2.E]

Mediciones con medir_tiempos.py contra el servicio desplegado en Render. La primera petición incluye el arranque en frío si el servicio estaba suspendido.

#	Petición	Código	Tiempo (ms)
1	GET /	[rellenar]	[rellenar]
2	GET /	[rellenar]	[rellenar]
3	POST /predict	[rellenar]	[rellenar]
4	POST /predict (MedInc = 3.0)	[rellenar]	[rellenar]
5	POST /predict (Los Ángeles)	[rellenar]	[rellenar]
6	POST /predict (cuerpo inválido)	[rellenar]	[rellenar]

[PENDIENTE: copiar aquí la tabla que imprime medir_tiempos.py y comentar el arranque en frío de la petición 1]

6) Documentación del proceso [RA2.F]

Los pasos seguidos (creación de la cuenta, configuración del repositorio, comandos de build y de despliegue y URL final) están documentados en README.md, incluido en el ZIP y en el repositorio de GitHub.

7) Principios éticos y de seguridad [RA2.G]

El dataset California Housing no contiene datos personales sensibles: cada fila es una zona censal con valores agregados (ingreso mediano, edad media de las viviendas, población, ocupación media y coordenadas de la zona), no una persona ni una vivienda concreta, y no incluye nombres, direcciones ni identificadores, por lo que el riesgo de reidentificación es muy bajo. El problema está en los sesgos: el modelo se apoya sobre todo en el ingreso y en la latitud y longitud, que pueden funcionar como indicadores indirectos del nivel socioeconómico o de la composición del barrio, de modo que podría asignar sistemáticamente precios menores a ciertas zonas y reforzar desigualdades existentes. Además, los datos proceden del censo de 1990 y el precio está limitado a 500.000 $ (992 zonas), por lo que el modelo no representa el mercado actual y infravalora las zonas más caras. No debería usarse para decisiones sobre personas (créditos, seguros, tasaciones) sin auditar antes su error por zona y sin dejar claros sus límites.

8) Integración de herramientas cloud [RA2.H]

app.py descarga modelo_california.pkl desde GitHub (URL raw) cuando no lo encuentra en local al arrancar:

```
MODEL_URL = os.getenv(
    "MODEL_URL",
    "https://raw.githubusercontent.com/<usuario>/<repositorio>/main/modelo_california.pkl",
)

def download_model(url: str | None = None, path: str = MODEL_PATH):
    url = url or MODEL_URL
    response = requests.get(url, timeout=DOWNLOAD_TIMEOUT)
    response.raise_for_status()
    with open(path, "wb") as f:
        f.write(response.content)

def load_model(path: str = MODEL_PATH):
    if not os.path.exists(path):
        print(f"{path} no encontrado en local, descargando...")
        download_model()
    return joblib.load(path)
```